

PRD — Flashpoint Desktop for Raspberry Pi

Section I — Product PRD / Semantic Product Definition

1. Product Name

Flashpoint Desktop

Working description:

A map-first unrest and disruption monitoring workstation that runs as a dedicated desktop application on a Raspberry Pi touchscreen.

2. Product Vision

Flashpoint Desktop is a **serious, always-on analyst workstation** for monitoring unrest-related public signals in the United States. It should feel like a real piece of installed equipment: persistent, stable, touch-friendly, fast to open, and cleanly self-contained.

The product is not meant to feel like:

- a browser tab
- a hobby dashboard
- a generic news map
- a fake military prop

It is meant to feel like a **single-purpose operational workstation** that sits on a desk and is always ready.

That is consistent with the original Flashpoint vision: a map-first, local-first, source-backed U.S. flashpoints monitor built for one operator, with a restrained tactical feel rather than theatrical clutter.

3. Product Thesis

The product should answer one simple emotional and functional need:

“I want this to feel like a real device, not a website I happen to open on a Pi.”

Today, Flashpoint is already a credible local MVP with:

- a FastAPI + SQLite backend
- a React/Vite + MapLibre frontend
- hotspot computation
- system freshness
- ingest run tracking
- operator status UI
- a map-first multi-panel interface.

The next product leap is not merely more dashboard polish. It is a **delivery-form shift**:

from:

- local web app + kiosk thinking

to:

- installed Pi application + appliance thinking

4. End-State User Experience

A user powers on the Pi. The device boots into the Flashpoint application automatically. There are no browser tabs, URL bars, terminal windows, or visible app launch steps.

Within a short period after boot:

- Flashpoint opens fullscreen
- the system restores into a usable operational view
- current freshness/status is visible
- map interaction is responsive
- the app remains on screen continuously
- if the backend is starting or offline, the app communicates that cleanly without crashing

The device should feel like a **purpose-built intelligence workstation**, not a general desktop computer.

5. Core Product Principles

These principles stay true to the original PRD, but are adapted for the desktop-app direction.

Map first

The center of gravity remains the operational map.

Installed-software feel

The app should feel native, dedicated, and persistent.

Truthful operational state

The app must communicate freshness, stale data, and backend failure honestly.

Touch-first practicality

The UI should work on the Raspberry Pi Touch Display 2 in portrait orientation without tiny controls or desktop-only assumptions. The original hardware target remains Pi 5 + Touch Display 2, portrait, touch input, local fullscreen workstation.

Local-first resilience

The device must keep working gracefully during temporary network loss, using last known data and persistent local state. The current technical PRD already requires last-known-data behavior and local failure persistence.

Tactical, not theatrical

The app should remain dark, restrained, premium, and operational.

6. What “Perfect Pi Software-Type Application” Means

For this PRD, “perfect” does not mean literally final forever. It means the first version that is **architecturally clean enough to be the real direction**, not a disposable hack.

That means Flashpoint Desktop must eventually be:

- a real desktop application shell
- bootable and launchable as the primary experience on the Pi
- stable across reboots
- free of visible browser chrome
- self-monitoring
- easy to install on a fresh Pi
- maintainable by Claude Code or a senior engineer without spaghetti deployment logic

7. Product Scope

In scope

Flashpoint Desktop includes:

- the existing Flashpoint operational map, feed, priorities, filters, detail, hotspot, and freshness experience
- a real desktop application shell

- backend runtime integration
- Pi boot/autostart behavior
- touch-friendly fullscreen operation
- local persistence and recovery behavior
- install/bootstrap flow for a fresh Pi
- clear temporary vs permanent architecture transitions

Out of scope

Not included in this PRD:

- multi-user accounts
- cloud sync
- collaboration
- mobile app
- handheld secondary terminal
- predictive forecasting
- people-level identification or surveillance
- a full AI chat interface

These remain aligned with the original PRD boundaries.

8. Product Modes: Temporary vs Permanent

A key requirement of this PRD is to distinguish between **temporary implementation steps** and **clean long-term renditions**.

Temporary transition layer

For speed and controlled migration, the first desktop version may:

- keep the existing FastAPI backend
- keep the existing React/MapLibre frontend for the map surface
- host the frontend inside the desktop app through an embedded web view
- communicate with the backend over localhost or a tightly controlled internal interface

This is a **deliberate transitional architecture**, not a random shortcut.

Permanent clean rendition

The mature Flashpoint Desktop should move toward:

- a true PySide6 application shell as the primary UI container
- React/web rendering only where it is still justified, especially for the map surface

- progressively more native desktop UI for chrome, panels, status surfaces, and lifecycle handling
- a cleaner application-runtime boundary where backend services are managed as part of the product, not as an ad hoc dev server living beside it

In plain language:

- **temporary** = “wrap and control what already works”
- **permanent** = “own the full application experience cleanly”

9. Product Milestones

Milestone A — Controlled Desktop Shell

Flashpoint runs as a fullscreen PySide6 application on desktop OS, with the current UI loaded inside an embedded web surface or hybrid shell.

This milestone is about replacing “browser kiosk” with “installed app.”

Milestone B — Operational Appliance Behavior

Flashpoint Desktop auto-launches on boot/login, recovers from backend startup delay, shows real status/failure states, and behaves like a trustworthy workstation.

Milestone C — Native Shell Maturation

The shell, status surfaces, loading states, settings, and operator controls become desktop-native. The web layer remains only where it is still the best fit.

Milestone D — Production Pi Packaging

A fresh Raspberry Pi can be provisioned, installed, launched, and rebooted into Flashpoint Desktop with minimal operator intervention.

This preserves the spirit of the original definition of done for the Pi platform layer: provision, reboot, and leave it on a desk to boot into a usable dashboard that ingests, stores, computes, and renders locally.

10. User Stories

Primary

I power on Flashpoint and it is already there, fullscreen, persistent, and ready.

Operational

I can tell whether the data is fresh, stale, syncing, or failed without opening logs.

Touch interaction

I can use the screen directly to inspect hotspots, filter the map, and read detail.

Recovery

If the backend is slow or a feed fails, the application stays calm and usable instead of crashing or exposing technical mess.

Installability

A future version of me, or someone helping me, can install this on a new Pi without reverse-engineering the repo.

11. Definition of Done for This PRD

This PRD is satisfied when Flashpoint is no longer “a website in kiosk mode that happens to run on a Pi,” but instead:

- launches as a dedicated desktop application
- feels persistent and installed
- hides browser identity completely
- manages its runtime cleanly
- survives reboot and restart
- preserves the current Flashpoint analyst experience
- remains local-first and Pi-appropriate

Section II — Technical PRD / Engineering Specification

12. Current State

Current repo state is materially strong and should be preserved, not discarded.

Implemented today:

- FastAPI backend
- SQLite local database
- React/Vite frontend
- MapLibre map
- event and hotspot APIs

- hotspot/trend computation
- system status/freshness endpoint
- ingest run persistence
- failure-aware operator status UI.

This means the project is **not** starting from scratch. The desktop pivot should be framed as a **runtime and shell evolution**, not a wholesale rebuild.

13. Architecture Decision

Chosen target architecture:

PySide6 desktop application shell + local Flashpoint services + selective embedded web rendering.

The rationale:

- Raspberry Pi kiosk mode officially supports booting into a full-screen application, not only a web page.
- Qt for Python provides [QWebEngineView](#), which can embed live web content and is therefore well-suited to preserving the current map/frontend investment while migrating toward a real desktop shell.
- The original technical PRD explicitly allowed a “local desktop app shell if justified,” even though the earlier V1 preference was Chromium kiosk.

14. Non-Goals of the Technical Transition

This transition should **not**:

- rewrite the entire frontend into native Qt immediately
- discard the existing backend logic
- create two parallel permanent runtimes
- depend on Electron as the final architecture
- introduce cloud-first or container-heavy deployment

15. Final Architecture Overview

15.1 Logical Components

1. Desktop Shell

- PySide6 application
- fullscreen window
- startup/recovery UI

- native status overlays where justified
- boot/runtime orchestration
- 2. Operational UI Layer**
 - initially: existing React UI embedded in web view
 - later: selected shell/UI surfaces moved into native Qt widgets
 - map may remain embedded web view longer than the rest
- 3. Local Services Layer**
 - FastAPI API layer
 - scoring / clustering / scheduler logic
 - ingestion orchestration
 - app health/runtime endpoints
- 4. Local Storage Layer**
 - SQLite
 - config/settings
 - ingest runs
 - logs/cache if needed

15.2 Runtime Topology

Near-term transitional topology:

PySide6 Desktop App

- └─ native shell window/chrome
- └─ embedded QWebEngineView for Flashpoint UI/map
- └─ talks to local FastAPI backend

FastAPI Backend

- └─ events/hotspots/status APIs
- └─ scheduler + ingestion
- └─ SQLite

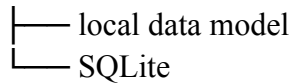
Longer-term mature topology:

PySide6 Desktop App

- └─ native shell surfaces
- └─ native lifecycle + settings + status
- └─ embedded map/web surface only where still justified
- └─ internal service boundary to Flashpoint runtime

Flashpoint Runtime

- └─ ingestion
- └─ scoring



16. Temporary vs Permanent Technical Strategy

16.1 Temporary Strategy — Transitional Hybrid

Purpose: achieve “installed software feel” quickly without discarding good work.

Characteristics:

- keep React frontend intact
- keep FastAPI intact
- launch a PySide6 fullscreen app
- embed the existing frontend using **QWebEngineView**
- route UI requests to localhost backend
- implement startup splash/loading/retry in native shell
- autostart the desktop app on Pi login

Why temporary:

- it still carries web UI infrastructure internally
- UI shell and map are not yet separated cleanly
- backend lifecycle is still more service-like than product-integrated

16.2 Permanent Strategy — Productized Desktop Runtime

Purpose: make the application’s runtime feel truly first-party and clean.

Characteristics:

- native Qt shell owns lifecycle, startup, errors, and operator chrome
- map remains embedded only if still best
- backend/runtime services are packaged as part of the application
- app settings, runtime health, logs, and install flow become productized
- deep reliance on browser assumptions is reduced over time

Why permanent:

- the product no longer conceptually depends on a browser-launch mindset
- operator-facing surfaces are native, deliberate, and maintainable

17. Proposed Repository Evolution

Temporary next-step structure

Flashpoint/
backend/
frontend/
desktop/
 app/
 main.py
 window.py
 shell/
 bridge/
 assets/
 requirements.txt
scripts/
deploy/
docs/

More mature structure

Flashpoint/
backend/
desktop/
 app/
 main.py
 window.py
 views/
 widgets/
 services/
 assets/
 state/
frontend/ # retained only for map/web surfaces if still justified
scripts/
deploy/
docs/

18. Desktop Application Requirements

18.1 Shell

The desktop app shall:

- open fullscreen by default
- hide browser identity entirely

- support touch interaction cleanly
- display startup/loading state
- display backend unavailable state
- display reconnect/retry state
- never expose raw Chromium tabs, URLs, or developer UI in normal operation

18.2 Windowing

The app shall:

- support Raspberry Pi OS desktop session
- support portrait orientation target
- avoid window chrome where possible
- relaunch cleanly on next login/boot if closed unexpectedly

18.3 Native Surfaces Required

Even in the hybrid phase, these should be considered shell-owned over time:

- startup screen
- backend connecting screen
- fatal error screen
- operator status ribbon
- settings/preferences
- update/install diagnostics

19. Map and UI Rendering Strategy

Transitional decision

The current Flashpoint map and multi-panel dashboard may remain embedded in `QWebEngineView` initially. Qt officially supports loading web content in `QWebEngineView`.

This is acceptable because:

- it preserves MapLibre work
- it minimizes risk
- it gives the app installed-software feel immediately

Permanent direction

Over time:

- desktop shell surfaces become native Qt widgets
- the map may remain web-rendered if that remains the most efficient and stable option
- the final architecture does **not** require that every pixel be native, only that the product experience be application-first

20. Backend Strategy

Short-term

Retain the current FastAPI backend as-is, with targeted cleanup only where needed:

- explicit serialization cleanup
- stable health/readiness endpoints
- scheduler hardening
- ingestion evolution from mock to real feeds

Medium-term

Refactor backend into a clearer runtime package boundary:

- `services/ingestion`
- `services/scoring`
- `services/runtime`
- `services/status`
- `storage`

Long-term

Permit one of two permanent models:

- FastAPI remains as internal/local API boundary because it provides clean separation
- or non-UI services are imported directly into the desktop runtime where that simplifies reliability

This decision can be deferred. The PRD should not force premature collapse.

21. Boot and Launch Flow

Transitional launch flow

1. Raspberry Pi boots into desktop session
2. Pi auto-login occurs
3. Flashpoint Desktop autostarts as the primary application

4. Native shell appears immediately
5. Shell waits for backend readiness
6. Once ready, shell loads operational UI
7. If backend is unavailable, shell shows native retry/recovery state

This is better aligned with current Raspberry Pi guidance than a brittle system service that guesses display variables for Chromium. Raspberry Pi's current kiosk guidance uses desktop-session autostart under `~/.config/labwc/autostart`, and explicitly describes kiosk mode as able to launch a full-screen application.

Permanent launch flow

Same flow, but with more of the runtime supervised and productized by the desktop app itself.

22. Service Management

Required near-term services

At minimum:

- backend runtime service
- optional scheduler under backend process or dedicated service
- desktop app autostart entry

Transitional service split

Recommended:

- backend managed by `systemd`
- desktop app launched via desktop autostart on user session login

This avoids forcing GUI processes through system services.

Permanent service model

One of:

- backend remains a managed background service and desktop app is the foreground client
- desktop app supervises internal runtime child processes cleanly

Either is acceptable if reliability is preserved.

23. Install and Provisioning Requirements

The install flow must move from “developer bring-up” to “product provisioning.”

Temporary provisioning

A scripted bootstrap may:

- install Python and venv deps
- install Node deps and build frontend assets
- install desktop runtime deps
- install/update systemd service
- install/update desktop autostart entry
- validate backend health
- validate app launch prerequisites

Permanent provisioning

A cleaner install flow should eventually:

- provision a fresh Pi with minimal manual steps
- create a reproducible desktop-runtime install
- validate graphics/web engine support
- validate touch display configuration
- validate autostart and recovery behavior
- document update/rollback flow

24. Hardware and OS Constraints

The target remains the original hardware/platform target:

- Raspberry Pi 5
- Raspberry Pi Touch Display 2
- portrait orientation
- active cooling
- Raspberry Pi OS 64-bit with desktop
- local fullscreen workstation deployment.

25. Performance Requirements

The original PRD’s performance framing still applies:

- cold boot to usable dashboard under 90 seconds
- dashboard initial load after launch under 8 seconds
- map interaction latency under 250 ms in normal state

- detail panel open under 500 ms
- 30-minute ingest cycle under 5 minutes under normal feed volume
- no thermal shutdown or normal-operation UI freeze.

For the desktop-app pivot, add:

- native shell visible within 10 seconds of desktop session start
- backend connection feedback visible immediately
- recovery from backend restart without requiring user relaunch
- no visible browser chrome ever in production mode

26. Offline and Failure Behavior

The original platform requirements remain binding:

- show last known data on feed failure
- stale-data indicator when freshness exceeds threshold
- persist ingestion failures locally
- no crash on temporary network loss.

Additional desktop-app requirements:

- if backend not ready, app stays open and communicates status
- if embedded web surface fails to load, app shows fallback shell state
- if scheduler fails, app remains usable for inspection
- app must not dump the user to the desktop on recoverable failures

27. Logging and Diagnostics

The original PRD already requires local logging for ingestion, API errors, scoring errors, UI launch failures, and startup/health checks.

Desktop-app additions:

- shell launch logs
- backend readiness timing
- web view load failures
- crash/restart diagnostics
- autostart success/failure diagnostics

28. Security and Operational Hardening

For a dorm/desk appliance version:

- run local-only by default unless explicitly enabled
- prefer loopback for local backend comms
- no exposed remote admin by default
- optional SSH-only maintenance for operator
- clearly documented update path

For later unattended or semi-public deployments:

- session hardening
- physical screen/power resilience
- optional overlay/read-only filesystem strategy
- credential hygiene for any future APIs

29. Migration Phases

Phase 1 — Desktop Shell Bootstrap

Deliverables:

- **desktop/** app scaffold
- PySide6 fullscreen shell
- embedded Flashpoint UI
- local backend connection
- startup/loading state
- manual dev launch on Mac and Pi

This is the first **temporary but intentional** phase.

Phase 2 — Pi Runtime Integration

Deliverables:

- Pi autostart flow
- backend systemd service
- desktop app autostart entry
- boot/recovery validation
- portrait/touch tuning
- screen blanking and idle behavior control

Phase 3 — Native Operator Shell

Deliverables:

- native startup screen
- native runtime state presentation
- native settings/diagnostics surface
- tighter control over connection/recovery lifecycle

This is where the product starts looking clean, not merely wrapped.

Phase 4 — Runtime Cleanup

Deliverables:

- cleaner boundary between shell and backend
- optional partial absorption of shell-adjacent UI into Qt
- reduced web-only assumptions
- reproducible install/update tooling

Phase 5 — Production Pi Appliance

Deliverables:

- documented fresh-Pi installation path
- reboot-tested app lifecycle
- resource tuning and thermal validation
- operator-ready reliability

30. Acceptance Criteria

Transitional success criteria

The transition is successful when:

- Flashpoint opens as a real desktop app
- no Chromium kiosk window is visible
- current dashboard functions still work
- backend status/freshness still work
- app auto-launches on Pi login
- app survives reboot cleanly

Permanent success criteria

The platform is truly successful when:

- the desktop shell is the canonical product runtime
- installation is reproducible

- the app is operationally trustworthy
- a fresh Pi can be turned into a Flashpoint workstation with bounded, documented setup effort
- the product clearly feels like “a real analyst tool,” not “a pretty map” and not “a browser kiosk.” That remains aligned with the original MVP success metric.

31. Engineering Directives for Claude Code / Senior Dev

- preserve the current backend investment
- do not rewrite everything at once
- use tightly scoped milestones
- distinguish transitional architecture from permanent architecture explicitly in code and docs
- do not introduce accidental permanent shortcuts
- avoid parallel runtimes that create confusion
- make install and boot flows reproducible
- prefer reliability over novelty
- keep the interface restrained and operator-first

32. Immediate Next Build Step

The next implementation step after adopting this PRD should be:

Build Phase 1: Flashpoint Desktop Shell Bootstrap

That means:

- create **desktop/** scaffold
- stand up PySide6 fullscreen app
- embed current Flashpoint UI in a controlled web view
- connect to local backend
- implement native loading/unavailable shell states
- keep the current repo working during the transition

That gives you the first real shift from “website project” to “software product.”